Ders 20 Genişletilmiş Notu: Uygulama: İkinci Ödevin Kontrolü ve Çift Hash ile Hash Tablosu Çözümü

Ders 20 Genişletilmiş Notu: Uygulama: İkinci Ödevin Kontrolü ve Çift Hash ile Hash Tablosu Çözümü

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Oturumun Yapısı
- 2. Problem: Kelimeleri Dosyalarıyla Birlikte İndekslemek
- 3. Veri Yapısı: Tablo Boyutları ve Tablo Gözü
- 4. Eski Tabloyu Dosyadan Okumak
- 5. Ana Akış ve Doluluk Oranı Kontrolü
- 6. Ekleme Fonksiyonu
- 7. Arama Fonksiyonu
- 8. Tablonun Dosyaya Kaydedilmesi
- 9. Örnek Sorular
- Dersin Toparlaması

1. Oturumun Yapısı

Bu kayıt, 9 Aralık 2020 Çarşamba gününün ikinci kaydıdır ve bir uygulama oturumudur. İki şubenin öğrencileri aynı oturumda toplandı. Yeni bir konu anlatılmadı. Oturumun ilk bölümünde (yaklaşık 00:05 ile 01:16 arası) ikinci ödevin kontrolü yapıldı. Kaydın bu kısmı Zoom odalarının ayarlanması ve ses sorunlarıyla ilgili konuşmalardan oluşur, akademik içerik taşımaz.

İkinci bölümde (yaklaşık 01:16 ile 01:37 arası) bir asistan ödevin çözümünü sözde kod (pseudocode) üzerinden anlattı. Asistan C kodunu değil, algoritmanın nasıl olması gerektiğini anlatmak istediğini özellikle belirtti. Bu notun geri kalanı bu çözüm anlatımını izler.

> **Hocanın vurgusu:** Hoca, ödev çözümlerinin asistanlar tarafından titizlikle hazırlandığını, her yıl öğrencilerin çözümleri istediğini ve bu anlatıma herkesin katılmasının karşılıklı kazancı en yükseğe taşıyacağını söyledi.

2. Problem: Kelimeleri Dosyalarıyla Birlikte İndekslemek

Elimizde bir takım dosyalar ve bu dosyaların içinde kelimeler var. Bir kelimenin hangi dosyalarda geçtiğini bulmak istiyoruz. Bu aramayı doğrudan dosyaların içinde yapmak ciddi biçimde uzun sürer. Bunun yerine kelimeleri bir hash tablosuna (hash table) yazıp indekslersek, bir kelimeyi aramak yalnızca sabit zaman (constant time) alır.

Asistan, hocanın ödevde istediği bazı ayrıntıları da hatırlattı:

- Doluluk oranının (load factor) tutulması ve yazdırılması.
- Kelimenin (string) Horner yöntemiyle sayısal bir anahtara (key) çevrilmesi.
- Çakışmaların (collision) çift hash (double hashing) ile çözülmesi.
- Aramanın büyük-küçük harfe duyarsız (case insensitive) olması.

> **Ek bilgi (derste anlatılmadı):** Sabit zaman, büyük olasılıkla ortalama durum için söylenmiştir. Açık adreslemede (open addressing) beklenen deneme sayısı doluluk oranı α büyüdükçe artar; başarısız bir arama için yaklaşık $1/(1-\alpha)$ denemedir. Doluluk oranı 1'e yaklaştıkça bu sayı hızla büyür. Ödevdeki 0.8 uyarı eşiği de büyük olasılıkla bu yüzden konmuştur.

3. Veri Yapısı: Tablo Boyutları ve Tablo Gözü

3.1 İki boyut değeri

Çözümde iki sabit vardır. M_{h1} tablonun uzunluğudur, yani göz sayısıdır. M_{h2} ise çift hash'te bir sonraki bakılacak indeksin ne kadar uzakta olduğunu, yani adım boyunu (step size) hesaplamakta kullanılan ikinci değerdir. Asistanın gösterdiği PDF'de bu değerler $M_{h1} = 997$ ve $M_{h2} = 996$ olarak yazılıydı (kısa notta kareden okunduğu belirtilmiştir).

Asistan, tablo boyutunun hocanın ödevde verdiği bir ayrıntı olduğunu söyledi: bine en yakın asal sayı. Bin'den sonraki asalı o an hatırlayamadığını, ama en yakının 997 olduğunu hatırladığını belirtti.

> **Ders sonrası not:** Asistanın hatırladığı doğrudur. 1000 civarındaki asallar 991, 997, 1009 ve 1013'tür (python ile doğrulandı). 997'nin 1000'e uzaklığı 3, 1009'un uzaklığı 9'dur. Yani 997 hem 1000'den küçük en büyük asal hem de 1000'e en yakın asaldır.

3.2 Tablo gözü: item

Tablonun her gözüne bir öğe (item) yazılır. Asistan, okuduğu ödevlerin tamamında bunun bir yapı (struct) ile yapıldığını söyledi. Bir öğenin üç alanı vardır:

- **index:** Kelime hash fonksiyonundan geçirildiğinde elde edilen indeks.
- **value:** Kelimenin (word) kendisi.
- **fileName:** Kelimenin geçtiği dosya adları. Bir kelime birden fazla dosyada geçebileceği için burada birden fazla ad tutulur.

Dosya adlarını tutmak için öğrenciler farklı yollar seçmişti. Bazıları bağlı liste (linked list), bazıları dinamik bellek ayırma (dynamic allocation), bazıları statik bir dizi (array) kullanmıştı. Hepsi doğru kabul edildi. Asistan kendi görüşüne göre bağlı listenin en uygun yapı olduğunu, ama diğerlerinin de tercih meselesi olduğunu söyledi.

Ardından tablo için bellek ayrılır: her göz bir öğe boyutunda olmak üzere M_{h1} tane göz. PDF'deki ilk satırlar şöyleydi (kısa nottaki kare okumasından):

```
Mh1 <- 997
Mh2 <- 996
item <- (item.index, item.value, item.fileName[MAXNUMBEROFFILE])
allocate hashTable[sizeof(item) * Mh1]
```

4. Eski Tabloyu Dosyadan Okumak

Program ilk kez çalışmıyorsa, yani daha önce bir hash tablosu oluşturulup bir dosyaya kaydedildiyse, tablo o dosyadan okunur. Hocanın ödevde ipucu olarak verdiği PDF'e göre son doluluk oranı da aynı dosyadan okunup güncel doluluk oranı olarak atanır.

```
if read Stored Hash Table from file
    hashTable  <- oldHashTable
    LoadFactor <- lastLoadFactor
```

Asistan dosya okumanın C ayrıntılarına girmedi. Öğrencilerin dosya okumayı az çok bildiğini söyleyip algoritmaya odaklandı.

5. Ana Akış ve Doluluk Oranı Kontrolü

Dosyadan kelimeler okunur ve her kelime `insertHashTable` adlı bir fonksiyonla tabloya eklenir. Bu fonksiyon o ana kadar oluşturulmuş tabloyu, okunan kelimeyi, dosya adını, doluluk oranını

ve iki boyut değerini (asal tablo boyutu ve adım boyu için kullanılan değer) alır. Güncellenmiş doluluk oranını döndürür.

Ekleme, doluluk oranına göre üç aşamada yürür:

- **Doluluk oranı 0.8'den küçükse:** Kelimeler normal biçimde eklenir.
- **Doluluk oranı 0.8 ile 1 arasındaysa:** Ekleme sürer, ama her eklemede güncel doluluk oranı yazdırılır ve bir uyarı mesajı verilir.
- **Doluluk oranı 1 olduysa:** Artık ekleme yapılmaz, hata verilir.

```
#INSERTION
while read word from file and LoadFactor < 0.8:
    LoadFactor <- insertHashTable(hashTable, word, fileName, LoadFactor, Mh1, Mh2)

while read word from file and LoadFactor >= 0.8 and LoadFactor < 1.0:
    LoadFactor <- insertHashTable(hashTable, word, fileName, LoadFactor, Mh1, Mh2)
    display warning and current load factor

if LoadFactor >= 1.0
    display error
```

6. Ekleme Fonksiyonu

6.1 İlk aday indeks

Asistan, çift hash formülündeki deneme sayacı i'yi "rotation" diye adlandırdı. Sıfırıncı denemede şu adımlar yapılır:

1. Gelen kelime küçük harfe çevrilir. Çünkü aramanın büyük-küçük harfe duyarlı olmaması istenmişti. Küçük harfli kelime yeni öğenin value alanına yazılır.
2. Kelime Horner yönteminden geçirilir ve bir anahtar (key) elde edilir.
3. Anahtar birinci hash fonksiyonundan (h_1) geçirilir ve bir indeks elde edilir. Yeni öğenin indeksi önce bu h_1 değeri olarak düşünülür.
4. Dosya adı da yeni öğenin dosya adı alanına yazılır.

En iyi durumda (best case) kelimenin bu ilk indekse yerleşmesini bekleriz. Ama her zaman böyle olmaz.

> **Ek bilgi (derste anlatılmadı):** Horner yöntemi, L karakterli bir kelimeyi key $= \sum_{k=0}^{L-1} c_k R^{L-k-1}$ biçiminde bir sayıya çevirir. Burada c_k karakter kodu, R sabit bir tabandır. Hesap key $\leftarrow$ key $\cdot\, R + c_k$ adımlarıyla yapılır, böylece üs alma gerekmez. Uzun kelimelerde bu sayı tam sayı sınırını aşabilir; bu yüzden büyük olasılıkla her adımda mod alınması ya da geniş bir tam sayı tipi kullanılması gerekir.

6.2 Çakışma ve çift hash döngüsü

Çakışma, kelimenin gideceği indekse daha önce başka bir kelimenin yazılmış olmasıdır. Bu durumda adım boyu kullanılarak bir sonraki indekse bakılır. Orası da doluysa aynı şekilde ilerlenir.

Döngüye girme koşulu üç parçanın aynı anda sağlanmasıdır:

- İndeksteki göz boş değildir.
- Gözdeki kelime aradığımız kelime değildir.
- Deneme sayacı hâlâ tablo boyutundan küçüktür, yani step size kadar bir adım daha atacak yer vardır.

Bu üçü birden sağlandığında adım boyu ikinci hash fonksiyonuyla (h_2) hesaplanır, yeni bir indeks oluşturulur ve sayaç bir artırılır. Asistan, her seferinde döngü başına dönüp koşullara yeniden bakıldığını vurguladı: Burası boş mu? Değil. İçinde istediğimiz kelime var mı? Yok. Sayaç daha ilerleyebilir mi? Evet. O zaman devam edilir. Çift hash dediğimiz yöntem budur.

Çift hash'te i. denemedeki indeks şöyledir:

$$\text{index}_i = (h_1(\text{key}) + i \cdot h_2(\text{key})) \bmod M_{h1}$$

Asistan h_2 için "şu ikiyi bu ikinci şeye göre moduna alıyoruz" dedi; yani anahtar M_{h2}'ye göre moda alınıyor.

PDF'deki çakışma döngüsü ve çıkış kontrolü şöyleydi (kısa nottaki kare okumasından):

```
#COLLISION VARSA
while hashTable[newItem.index] != NULL and
      strcmp(hashTable[newItem.index].value, newItem.value) != 0 and
      rotation < Mh1:
    stepSize <- hashFunction2(key, Mh2)
    newItem.index <- hashFunction(h1, stepSize, rotation, Mh1)
    rotation++

if hashTable[newItem.index] == NULL:          #yeri boş
    hashTable[newItem.index] <- newItem
    LoadFactor <- LoadFactor + (1/Mh1)
else if strcmp(hashTable[newItem.index].value, newItem.value) == 0:   #önceden kayıtlarda mevc
    append fileName hashTable[newItem.index].fileName[]
```

> **Ders sonrası not:** h_2 hiçbir zaman 0 vermemelidir. Yalnız key mod M_{h2} kullanılırsa, anahtar 996'nın katı olduğunda adım boyu 0 olur ve hep aynı göze bakılır. Yaygın çözüm büyük olasılıkla $h_2(\text{key}) = 1 + (\text{key} \bmod M_{h2})$'dir. Bu, 1 ile 996 arasında bir adım verir. $M_{h1} = 997$ asal olduğu için 1 ile 996 arasındaki her adım boyu tablonun 997 gözünün hepsini dolaşır (python ile doğrulandı). Asal olmayan bir boyutta, örneğin 1000'de, adım boyu 2 ise yalnız 500 göz dolaşılır.
>
> **Ders sonrası not:** PDF'deki döngüde yeni indeks hesaplandıktan sonra sayaç artırılıyor. Sayaç 0'dan başlarsa ilk döngü adımında $h_1 + 0 \cdot h_2 = h_1$ olur ve aynı dolu göz bir kez daha hesaplanır (python simülasyonunda doğrulandı). Sayacı artırıp sonra indeksi hesaplamak ya da sayacı 1'den başlatmak bu gereksiz adımı önler.

6.3 Döngüden çıkış nedenleri

Asistan, döngüden sonra "while'dan çıkmamızı ne sağladı?" diye sorup üç durumu ayırdı:

- **Bulunan göz boş:** Yer boştur. Öğe oraya yazılır. Öğe yazıldığı için doluluk oranı güncellenir, yani $1/M_{h1}$ kadar artar.
- **Gözdeki kelime aranan kelimeye eşit:** Kelime daha önce kaydedilmiştir. Bu durumda yalnız yeni dosya adı eklenir: dosya adları nasıl tutuluyorsa araya ya da bağlı listenin sonuna.
- **Deneme sayacı tablo boyutuna ulaştı:** Asistan önce "tablonun dışına taşar" dedi, sonra düzeltti: Aslında taşmak değil, baştan diğer gözlere yeniden ulaşmak demektir. Yani bütün gözlere bakılmıştır. Üzerine yazma (overwrite) istenmediği için "hash tablosu dolu" denir ve fonksiyondan dönülür.

Fonksiyonun sonunda, doluluk oranı içeride güncellendiği için güncel doluluk oranı döndürülür. Ana akıştaki 0.8 ve 1 kontrolleri bu dönen değere göre yapılır.

7. Arama Fonksiyonu

Arama da eklemeyle neredeyse aynı adımları izler. Önce kelime, eklemede olduğu gibi büyük-küçük harf duyarlılığı olmasın diye küçük harfe çevrilir. Bunun nasıl yapılacağı öğrenciye bırakılmıştır. Sonra Horner'dan geçirilip anahtar, ardından ilk indeks elde edilir.

Asistan aramayı şöyle düşünmeyi önerdi: Önce kelimenin aslında olması gereken ilk yere bakarız. Orada değilse olması gereken ikinci yere bakarız ve böyle devam ederiz. "Burada değil, sonraki adıma hop atlıyoruz, orada mı? Hop atlıyoruz, orada mı?" Eklemeyle neredeyse aynı şeyi yapmamızın sebebi budur: Ekleme kelimeyi hangi sırayla yerleştirdiyse, arama da aynı sırayla bakar.

Döngü, indeksteki göz doluyken ve içindeki kelime aranan kelime değilken sürer. Döngüden çıkış nedenleri şunlardır:

- **Bulunması gereken göz boş:** Kelime tabloda yoktur. Kelime eklenmiş olsaydı bu boş göze ya da daha önceki bir göze yazılmış olurdu.
- **Deneme sayacı çok büyüdü:** Bütün hash tablosu taranmış ama kelime bulunamamıştır. Bu en kötü duruma (worst case) ulaşıldığı anlamına gelir.
- **Kelime eşleşti:** Döngü, "kelime farklı" koşulu sağlanmadığı için kırılmıştır. Aranan kelime gözdeki değere eşittir, yani kelime oradadır. Bu durumda kelimenin geçtiği dosyalar ("şu dosyanın içinde, bu dosyanın içinde") istenen biçimde yazdırılır ve fonksiyondan çıkılır.

  Ek bilgi (derste anlatılmadı): Boş gözde aramayı bitirme kuralı, tablodan silme yapılmadığı sürece doğrudur. Silme olsaydı, silinen bir göz zincirin ortasında boşluk açar ve arkasındaki kelimeler bulunamazdı. Bu yüzden silme yapılan açık adreslemeli tablolarda silinen gözler genelde "silindi" (tombstone) diye işaretlenir. Bu ödevde silme istenmediği için sorun yoktur.

8. Tablonun Dosyaya Kaydedilmesi

Bir öğrenci hash tablosunun metin dosyasının nasıl göründüğünü sordu. Asistan kendisi bir tablo oluşturmadığını, ama öğrencilerin ödevlerinden örnek verebileceğini söyledi.

Öğrencilerin çoğu tabloyu metin (text) dosyasına alt alta yazmıştı. Her satıra tablonun bir gözü yazılmıştı: aralarında birer boşluk bırakarak, örneğin sıfırıncı gözün satırında önce kelime, sonra da kelimenin geçtiği dosya adı (input.txt gibi). Bütün gözler yazıldığında 997 satır olur; indeksler sıfırdan başladığı için son satırın indeksi 996'dır.

Öğrenciler dosyayı iki farklı biçimde yazmıştı:

- **Bütün gözleri yazanlar:** Boş gözler için boş satır bırakarak 997 satırın tamamını yazıp sonra okumuşlardı.
- **Yalnız dolu gözleri yazanlar:** Her satırın başına gözün indeksini yazmışlardı (örneğin 835). Okurken bu sayının gösterdiği indekse satırın devamındaki bilgiyi yazmışlardı.

Asistan, tablonun ikili (binary) dosya olarak da yazılabileceğini belirtti. Her bir yapı (struct) dosyaya olduğu gibi yazılırsa, sonra dosyadan blok blok okunabilir.

9. Örnek Sorular

Bu derste ekranda gösterilen bir soru yoktur. Aşağıdaki sorular, çözüm anlatımından sonra öğrencilerin sözlü ya da sohbet (chat) üzerinden sorduğu sorulardır.

Soru 1 (derste sözlü, 01:31:09): Çakışma döngüsü (while) bir hata vermeyecek mi? Çakışma olduğunda bir uyarı yapılmalı mı?

> **Cevap (derste verildi):** Hayır. Çakışma bir hata olarak algılanmaz. Çift hash'te çakışma olması beklenir; çakışma olunca bir sonraki adımda başka bir göze bakılır.

Soru 2 (derste sözlü, 01:31:59): Tablodaki indeks NULL ise döngüden çıkıp "yeri boş" kısmına mı geliriz?

> **Cevap (derste verildi):** Göz baştan NULL ise döngüye hiç girilmez. Döngünün üç koşulu VE (and) ile bağlıdır; herhangi biri sağlanmazsa döngüye girilmez ya da döngüden çıkılır. Döngüden sonra çıkış nedenine bakılır: göz NULL ise öğe oraya yazılır.

Soru 3 (derste sözlü, 01:32:55): Hash tablosunun kaydedildiği metin dosyası nasıl görünüyor?

> **Cevap (derste verildi):** Her satırda tablonun bir gözü vardır: kelime ve geçtiği dosya adları, aralarında boşlukla. Tam tablo 997 satırdır (indeks 0 ile 996). Tablo ikili dosyaya yapı yapı da yazılıp blok blok okunabilirdi.

Soru 4 (derste sözlü, 01:35:16 - 01:36:30): Dosyadan okurken, örneğin 97. satıra (97. göze) nasıl gidilmiş? Bir kelimeyi ararken bütün satırlar mı geziliyor?

Cevap (derste verildi): Dosyada arama yapılmaz. Önce bütün tablo dosyadan okunup bellekteki hash tablosuna yazılır. Bütün gözleri yazanlar satırları sırayla okuyup aynı sıradaki gözlere yerleştirmiştir. Yalnız dolu gözleri yazanlar, satır başındaki indeks numarasını okuyup satırın devamını o indeksteki göze yazmıştır. Arama, bundan sonra bellekteki tabloda Horner ve hash fonksiyonlarıyla yapılır.

Dersin Toparlaması

Bu uygulama oturumunda ikinci ödevin çözümü, yani dosyalardaki kelimeleri geçtikleri dosya adlarıyla birlikte bir hash tablosunda tutan program, sözde kod düzeyinde anlatıldı. Tablo 997 gözlüdür, çünkü tablo boyutunun bine en yakın asal sayı olması istenmişti. Her göz bir kelime ve onun geçtiği dosya adlarının listesini tutar. Kelime önce küçük harfe çevrilir, Horner yöntemiyle anahtara dönüştürülür, h_1 ile ilk indeks ve h_2 ile adım boyu hesaplanır. Çakışmada adım boyu kadar ilerlenerek boş bir göz, aynı kelimeyi taşıyan bir göz ya da tablonun tamamen tarandığı durum aranır.

Arama, eklemenin izlediği yolun aynısını izler: Kelimenin olması gereken ilk yere, sonra ikinci yere bakılır. Boş göze rastlanırsa kelime tabloda yoktur. Doluluk oranı her yeni kelimede $1/M_{h1}$ kadar artar; 0.8'i geçince uyarı verilir, 1'e ulaşınca ekleme durur.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.